Introduction to factorials, permutations, and combinations

Jessica Taberner

Summary

The factorial function has many mathematical uses. It is key to many counting questions. This guide outlines what the factorial function is and how you can use it in order to calculate the number of permutations or combinations of things.

What are factorials, combinations, and permutations?

Factorials, combinations, and permutations are fundamental tools used to count arrangements and selections. Permutations are useful when order matters, for example, possible password options, or different outcomes in a race. Combinations are used when order does not matter, for example, winning lottery tickets, or counting the ways you could select people for a team. Factorials are used for rearranging all items, for example, the number of ways you could arrange 5 different jars of sweets on a shelf.

These ideas have many real-life applications. They're very useful within probability and game theory, often being needed to calculate the likelihood of specific outcomes. You may use these concepts in computer science or cybersecurity, when analyzing algorithms or encryption methods, or within data analysis when selecting samples from a large group.

Defining factorials

Factorials are needed for many counting questions. Within this guide you will see them used to calculate the number of ways you can order things from a collection, or chose things from a collection, but you may also see them used in broader contexts. Factorials are used in Taylor series expansions, power series, and binomial theorems.

> **Definition of factorial**
>
> If n is a positive integer, $n!$ is defined to be:

$$n! = n \cdot (n-1) \cdot (n-2) \cdots 2 \cdot 1$$

You can think of n factorial as describing the number of possible ways to order n things. Let's see this in action with an example.

Example 1

In the sweet shop, Cantor's Confectionery, how many ways can you list 5 types of sweets on a sign? Let's think of the 5 possible places.
When filling the first spot on the sign, there are 5 sweets you can pick from. But when you go to fill the second spot on the sign, there are now only 4 possible sweets you could select.
Repeating this process, you will have 3 choices for the third spot, 2 choices for the second spot, and only 1 choice for the last spot.
Since all of these choices are independent, meaning one choice wont effect the others, this gives you a total of $5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 120$ possible orderings. You can notice that this is $5!$.

Warning

You should note that $0!$ and $1!$ are special cases.

$$0! = 1$$

$$1! = 1$$

You can think of 0 factorial as describing the number of possible ways to order 0 things, which is defined to be 1.
Similarly $1! = 1$, as there is only 1 way to order 1 thing.

Example 2

How many ways could you order 9 different bags of sweets at Cantor's Confectionery? You can use the factorial function for this.

$$9! = 9 \cdot 8 \cdot 7 \cdot 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 362880$$

So there are 362880 ways to organize the 9 bags.

You can notice that when computing factorials, numbers can get large quickly!

Defining permutations

Maybe you want to count something more specific. Say you need to select an ordered collection of things from a larger group, for example, if there are 20 types of sweets sold at Lovelace's Lollies, but only 10 spots on the shelf, how many ways could you organize the sweets into these 10 spots?

i Definition of permutations

A **permutation** is a way of arranging r things from a collection with n elements, **where order matters**.
The **permutation formula** calculates the number of ways to do this.

$$P(n,r) = \frac{n!}{(n-r)!} = \frac{n \cdot (n-1) \cdot (n-2) \cdot \cdot 2 \cdot 1}{(n-r) \cdot (n-r-1) \cdot (n-r-2) \cdots 2 \cdot 1}$$

To use this formula n and r must be integers, with $r \leq n$. If $r > n$ then there is no way to permute r things from a collection of n things, so $P(n,r) = 0$.
You may see this formula used elsewhere called the **falling factorial**.

You might also see ${}_nP_r$ notation used for permutations in other texts.

Let's see why this formula works using a variation of the example above.

Example 3

If there are 14 types of sweets sold at Cantor's Confectionery, but only 5 spots on the shelf, how many ways could you organize the sweets into these 5 spots?
For the first spot you have 14 options to chose from, then for the second you have only 13. Repeating this process, you will have to choose from 10 possible options for the last spot on the shelf. Since all of these choices are independent, that is:

$$14 \cdot 13 \cdot 12 \cdot 11 \cdot 10 = 240240$$

Let's calculate this again using the permutation formula. This is asking you to calculate the number of ways of arranging 5 things from a 14 element collection, where order matters.

$$P(14,5) = \frac{14!}{(14-5)!} = \frac{14!}{9!} = \frac{14 \cdot 13 \cdot 12 \cdots 2 \cdot 1}{9 \cdot 8 \cdot 7 \cdots 2 \cdot 1}$$

If you simplify this fraction by canceling the elements that occur in both the numerator and denominator, you will get the following sum:

$$14 \cdot 13 \cdot 12 \cdot 11 \cdot 10 = 240240$$

Matching what you calculated above!

Let's see another example of this formula in action.

Example 4

Say there are 5 members of staff employed at Cantor's Confectionery. For a given shift at the store you need 3 workers, you want to select someone to work as a cashier, someone to work in the stockroom, and someone to work as a sales assistant.
The different roles make this a case of selecting a subset from a larger collection **where order matters**.
You need to arrange 3 employees from a group of 5.

$$P(5,3) = \frac{5!}{(5-3)!} = \frac{5!}{2!} = \frac{5 \cdot 4 \cdot 3 \cdot 2 \cdot 1}{2 \cdot 1} = \frac{120}{2} = 60$$

So, there are 60 ways to select 3 employees from a collection of 5 for these jobs.

Defining combinations

So far you have considered counting possible orderings of either the whole collection or a subset of the collection, but what about when order doesn't matter? Perhaps you need to select three employees, without assigning specific roles, or you want to count the different mixtures of pick-and-mix sweets you could have. For this, you'll need combinations.

i Definition of combinations

A **combination** is a way of selecting k things from a collection of n things, **where order doesn't matter**.
The **combination formula** calculates the number of ways to do this. You can write this in two ways:

$$C(n,k) = \binom{n}{k} = \frac{n!}{(n-k)!k!}$$

$$C(n,k) = \binom{n}{k} = \frac{n \cdot (n-1) \cdots (n-k+1)}{k \cdot (k-1) \cdots 2 \cdot 1}$$

You would read this as "n choose k".
To use this formula n and k must be integers, with $k \leq n$. If $k > n$ then there is no way to select k things from a collection of n things, so $\binom{n}{k} = 0$.

You can refer to this as the **binomial coefficient**. The reason for this name will be explained in **[Guide: Introduction to the binomial theorem.]**

You may also see ${}_nC_k$ notation used for combinations elsewhere.

Let's see why this formula works using a variation of the example above.

Example 5

Say there are still 5 members of staff employed at Cantor's Confectionery. For a given shift at the store you want 3 workers, but you aren't interested in assigning specific roles this time, so the order you select them in doesn't matter and you need to forget the ordering from the last example.

You need to select 3 employees from a group of 5.

In selecting the first employee, you have 5 choices, for the second you have 4, and for the third you have 3. This gives you $5 \cdot 4 \cdot 3 = 60$ options, but this is too many.

With this you will have overcounted the options. Say the first employee you select is Abby, the second is Ben, and the third is Charlie. This is no different than if you picked Ben first, then Charlie, then Abby, or Charlie first, then Ben, then Abby. You will have counted each combination 6 times! You can notice that this is $3!$, the number of possible orderings of three elements.

So you must divide the 60 permutations by 6 to fix the overcounting:

$$60 \div 6 = 10$$

So, there are 10 ways to choose 3 employees from a group of 5.

Let's repeat this using the formula.

$$\binom{5}{3} = \frac{5!}{(5-3)!3!} = \frac{5!}{2!3!} = \frac{120}{2 \cdot 6} = \frac{120}{12} = 10$$

Matching your first calculation!

Let's see another example of this.

Example 6

There are 15 different types of sweets at Lovelace's Lollies pick-and-mix stand. Customers need to pick 6 types to fill a medium bag. How many ways are there to do this?

You need to select 6 items from a group of 15, this is 15 choose 6. $15!$ is a 13 digit number, so this time, let's use the second version of the formula.

$$\binom{15}{6} = \frac{15 \cdot (15-1) \cdots (15-6+1)}{6 \cdot (6-1) \cdots 2 \cdot 1}$$

$$\binom{15}{6} = \frac{15 \cdot 14 \cdots 10}{6 \cdot 5 \cdots 2 \cdot 1} = \frac{3603600}{720} = 5005$$

So there are 5005 ways to chose your medium pick-and-mix bag!

Example 7

For the large pick-and-mix bag, customers can to pick 9 sweets to fill their bag, how many ways are there to do this?

You need to select 9 items from a collection of 15, this is 15 choose 9.

$$\binom{15}{9} = \frac{15 \cdot (15-1) \cdots (15-9+1)}{9 \cdot (9-1) \cdots 2 \cdot 1}$$

$$\binom{15}{9} = \frac{15 \cdot 14 \cdots 7}{9 \cdot 8 \cdots 2 \cdot 1} = \frac{1816214400}{362880} = 5005$$

So there are also 5005 ways to choose your large pick-and-mix bag!

Now you can spot an interesting symmetry within this formula. $\binom{15}{6} = \binom{15}{9}$, but why? You can understand this by considering the sweets you don't select. If you want to consider all the ways you can pick 6 sweets from 15 options, you could think of this as all the ways you could **not** pick 9 types of sweets.

You can generalize this to the following identity:

$$\binom{n}{k} = \binom{n}{n-k}$$

This symmetry can be seen best in **pascal's triangle.** This is named after the 17th-century French mathematician Blaise Pascal, but the triangle itself is far older.

Definition of pascal's triangle

Pascal's triangle is an infinite triangular array of binomial coefficients.
You can construct the triangle using a recursive method.
In row 0, there is a single 1. The entries in the following rows are constructed by adding the number above and to the left with the number above and to the right, treating blank entries as 0.

```
#| '!! shinylive warning !!': |
#|   shinylive does not work in self-contained HTML documents.
#|   Please set `embed-resources: false` in your metadata.
#| standalone: true
#| viewerHeight: 340
library(shiny)
library(bslib)

ui <- fluidPage(
  tags$head(
    tags$link(rel = "stylesheet", href = "https://fonts.googleapis.com/css2?family=Ro
    tags$style(HTML("
      body {
        font-family: 'Roboto', sans-serif;
        background-color: #f5f5f5;
        text-align: center;
      }
      .triangle-container {
        display: inline-block;
        text-align: left;
      }
      .triangle-row {
        display: flex;
        justify-content: flex-start;
        position: relative;
      }
      .triangle-number {
        width: 60px;
        height: 52px;
        background-color: #3f68b6; /* default blue */
        display: flex;
```

```
      justify-content: center;
      align-items: center;
      font-weight: bold;
      color: white;
      font-size: 16px;
      clip-path: polygon(
        50% 0%, 93% 25%, 93% 75%, 50% 100%, 7% 75%, 7% 25%
      );
      margin: -13px 0.5px 0 0.5px;
      transition: transform 0.2s, background-color 0.2s;
      cursor: default;
    }
    .triangle-number.hovered {
      background-color: #db4315; /* red on hover */
    }
    .triangle-number.parent {
      background-color: #ffcb00 !important; /* yellow for parents */
      color: black !important; /* black text for readability */
    }
    .triangle-row-label {
      width: 30px;
      text-align: right;
      font-weight: bold;
      font-size: 16px;
    }

    /* Custom slider styling */
    .js-irs-0 {color: #3f68b6; width: 100% !important; margin: 20px 0;}
    .irs-bar, .irs-bar-edge, .irs-line { background-color: #3f68b6 !important;}
    .irs-handle { color: #3f68b6; width: 15px; height: 15px; top: -9px; border: 2px
    .irs-single {
      background-color: #3f68b6;
      color: #3f68b6;
      font-weight: bold;
      font-size: 10px !important; /* bigger font */
    }

  "))
```

```
  ),

  titlePanel("Pascal's Triangle"),

  sidebarLayout(
    sidebarPanel(
      sliderInput("rows",
                  "Number of Rows:",
                  min = 1,
                  max = 10,
                  value = 5)
    ),
    mainPanel(
      div(class = "triangle-container", uiOutput("triangle_ui")),
      tags$script(HTML("
        // Delegated events for dynamically generated elements
        $(document).on('mouseenter', '.triangle-number', function() {
          var id = $(this).attr('id');
          $(this).addClass('hovered');
          var parents = $(this).data('parents');
          if (parents) {
            parents.forEach(function(pid){ if(pid) $('#' + pid).addClass('parent'); }
          }
        });
        $(document).on('mouseleave', '.triangle-number', function() {
          var id = $(this).attr('id');
          $(this).removeClass('hovered');
          var parents = $(this).data('parents');
          if (parents) {
            parents.forEach(function(pid){ if(pid) $('#' + pid).removeClass('parent')
          }
        });
      "))
    )
  )
)

# Generate Pascal's Triangle with parent IDs
```

```
generate_pascal_with_parent_ids <- function(n) {
  triangle <- vector("list", n)
  parent_ids <- vector("list", n)

  for (i in 1:n) {
    row <- numeric(i)
    row_parents <- vector("list", i)

    for (j in 1:i) {
      if (j == 1 || j == i) {
        row[j] <- 1
        row_parents[[j]] <- c(NA, NA)
      } else {
        row[j] <- triangle[[i-1]][j-1] + triangle[[i-1]][j]
        row_parents[[j]] <- c(paste0("hex_", i-1, "_", j-1),
                              paste0("hex_", i-1, "_", j))
      }
    }

    triangle[[i]] <- row
    parent_ids[[i]] <- row_parents
  }

  list(values = triangle, parent_ids = parent_ids)
}

server <- function(input, output, session) {

  output$triangle_ui <- renderUI({
    n <- input$rows
    tri_data <- generate_pascal_with_parent_ids(n)
    tri <- tri_data$values
    parents <- tri_data$parent_ids

    hex_width <- 60
    hex_spacing <- 0.5
    total_width <- n * (hex_width + hex_spacing*2)
```

```
    triangle_html <- lapply(seq_along(tri), function(i) {
      row <- tri[[i]]
      row_parents <- parents[[i]]

      row_divs <- lapply(seq_along(row), function(j) {
        div(id = paste0("hex_", i, "_", j),
            class = "triangle-number",
            `data-parents` = I(jsonlite::toJSON(row_parents[[j]])),
            row[j])
      })

      row_width <- length(row) * (hex_width + hex_spacing*2)
      offset <- (total_width - row_width)/2

      # Row container with label aligned with triangle
      div(
        style = "display: flex; align-items: center;",
        div(class = "triangle-row-label",
            style = paste0("margin-left:", offset, "px;"),
            paste0(i - 1)  # row label starting from 0
        ),
        div(
          class = "triangle-row",
          style = "margin-left: 5px;", # small space between label and hexes
          row_divs
        )
      )
    })

    do.call(tagList, triangle_html)
  })

}
shinyApp(ui = ui, server = server)
```

This allows you to establish another identity.

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$$

You can see these identities proved in Proof: Binomial identities

Since Pascal's triangle encodes the binomial coefficients, you may be able to use the triangle in your calculations. To find n choose k, go to the $k+1th$ place in the nth row of the triangle!

Counting strategies

Now that you've seen 3 key tools used in counting questions, let's see some different examples. There are some key questions to ask yourself when dealing with a counting question that will help you decide what technique will be the most useful.

Are you using everything in a group, or just a subset?

Does order matter or not?

Is repetition allowed?

Are there any extra conditions to keep in mind?

Example 8

For a display in Lovelace's Lollies, you want to arrange the 5 best-selling sweets in a circle. How many ways are there to do this?
Since order matters, you're using permutations:

$$5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 120$$

However, the extra condition of this being a circular arrangement makes 120 an over-count. Because a circle has no defined start or end, it can be rotated to different positions without changing the relative order of the items. You should divide by 5 to account for this:

$$120 \div 5 = 24$$

Example 9

Say you're getting a pick-and-mix bag at Cantor's Confectionery. There are 15 different types of sweets, 6 of which are bonbons. You want to pick 8 sweets, but you want this to include at least 3 different types of bonbons. How many ways are there to do this?

Since order doesn't matter, you would use combinations. To account for the fact that you want at least 3 bonbons, you should pick these out first. There are 6 bonbons to choose from:

$$\binom{6}{3} = \frac{6!}{3!3!} = \frac{720}{6 \cdot 6} = \frac{720}{36} = 20$$

So, there are 20 ways to chose your bonbons, how many ways are there to pick the final 5 sweets? You don't want to pick a sweet you already chose, so now there are $15 - 3 = 12$ sweets to pick from:

$$\binom{12}{5} = \frac{12 \cdot 11 \cdots 8}{5 \cdot 4 \cdots 2 \cdot 1} = \frac{95040}{120} = 792$$

So, there are 792 ways to pick your other 5 sweets.

Since you can pair any bonbon collection with any of your other sweets, you need to select 1 thing from a collection of 20 and 1 thing from a collection of 792.

$$20 \cdot 792 = 15840$$

Example 10

Say you're stocking the shelves at Lovelace's Lollies. There are 13 different types of sweets, including 6 kinds of jelly sweets. You want to arrange these sweets on the shelf, but you want to keep all the jelly sweets together. How many ways can you do this?

Since you want to keep the jelly sweets together, you should think of the 6 jelly sweets as 1 item, meaning you are considering the ways you can arrange $13 - 6 + 1 = 8$ items. This is $8!$:

$$8! = 8 \cdot 7 \cdots 2 \cdot 1 = 40320$$

But wait, how many ways could you arrange the collection of jelly sweets? This is $6!$:

$$6! = 6 \cdot 5 \cdots 2 \cdot 1 = 720$$

So, there are 720 ways of arranging the jelly sweets, and any one of these produces 40320 ways of arranging the rest of the sweets with them. So altogether there are $720 \cdot 40320 = 29030400$ ways of stocking this shelf!

You can notice that the jelly restriction has made a difference. Without the restriction there would be $13! = 6227020800$ arrangements!

Example 11

Now you want to stock 4 kinds of fudge and 5 kinds of chocolate, how many ways can you do this, making sure no 2 kinds of fudge are next to each other?

Let's first think of placing the 5 chocolates, leaving gaps, including at the ends. There are $5! = 120$ ways to do this. Now you have to fit the 4 kinds of fudge in the 6 gaps you created, this is $P(6, 4) = 360$.

Putting this together, there are $120 \cdot 360 = 43200$ ways to do this.

Quick check problems

1. How many ways could you order 6 different jars of jelly sweets at Lovelace's Lollies?
2. here are 15 different types of sweets at Cantor's Confectionery's pick-and-mix stand. Customers need to pick 4 types to fill a small bag, how many ways are there to do this?
3. If there are 10 types of bonbons sold at Lovelace's Lollies, but only 6 spots on the shelf, how many ways could you organize the bonbons into these 6 spots?

4. You are given three statements below. Decide whether they are true or false.

(a) Combinations are used when order matters.

(b) $0! = 0$.

(c) If two items must stay together, they can be treated as a single unit.

Further reading

For more questions on the subject, please go to Questions: Introduction to factorials, combinations, and permutations.

For more about binomial coefficients, please go to **[Guide: Introduction to the binomial theorem.]**

Version history

v1.0: initial version created 02/26 by Jessica Taberner as part of a University of St Andrews VIP project.

This work is licensed under CC BY-NC-SA 4.0.